

UCle

Universal Chiplet Interconnect Express

Final System for Mainband

Architecture & Design Specification

Eng : Ali Noureldin Abdelaziz

Revision 1.0

Contents

1. Format 3	3
1.1 Flit Header for Format 3	3
1.2 Flit Structure Description	4
2. MainBand Block	6
2.1 Mainband Block Overview	6
2.2 TX Path	8
2.3 RX Path	9
3. Packer	10
3.1 Overview	10
3.2 Signal Description	11
3.3 TX Path Operation	12
4. CRC Generator	13
4.1 Overview	13
4.2 Signal Description	14
4.3 CRC-16 Architecture (64 Bytes per Clock)	15
4.4 CRC Generator Operation	15
5. Unpacker	16
5.1 Overview	16
5.2 Signal Description	17
5.3 RX Path Operation	18
5.4 Flit Cancel Mechanism	18
6. CRC Check	19
6.1 Overview	19
6.2 Signal Description	20
6.3 CRC Check Operation	21

1. Format 3

1.1 Flit Header for Format 3

Table 1-1 defines the flit header structure applicable to Format 3, Format 4, Format 5, and Format 6 with Retry. The diagram below shows the complete header layout as specified in the UCle standard.

Table 3-5. Flit Header for Format 3, Format 4, Format 5, and Format 6 with Retry

Byte	Bit	Description			
		CXL 256B Flit Mode	PCIe Flit Mode	Streaming Protocol	Management Transport Protocol
0	[7:6]	Protocol Identifier: 00b: D2D Adapter/CXL.io NOP Flit 01b: CXL.io Flit 10b: CXL.cachemem Flit 11b: ARB/MUX Flit	Protocol Identifier: 00b: D2D Adapter/PCIe NOP Flit 01b: PCIe Flit All other encodings are reserved	Protocol Identifier: 00b: D2D Adapter NOP Flit Remaining encodings are permitted to be used by Protocol Layer in a vendor defined manner. Protocol Layer must never set this to 00b for Flits sent across FDI.	Protocol Identifier: 00b: D2D Adapter NOP Flit 01b: Management Flit All other encodings are reserved.
	[5]	Stack Identifier: 0: Stack 0 1: Stack 1			
	[4]	Reserved for CXL.cachemem For CXL.io or PCIe Flit Mode: 0: DLLP Payload in DLP 2..5 1: Optimized_Update_FC or Flit_Marker in DLP2..5		Reserved	
	[3:0]	The upper four bits of Sequence number "S" (i.e., S[7:4])			
	[7:6]	Flit Type: 00b: CXL/PCIe/Streaming Flit/D2D Adapter NOP Flit 01b: Test Flit (see Section 11.2 for details) 10b: Management Flit 11b: Reserved			

Figure 1-1: Flit Header for Format 3, Format 4, Format 5, and Format 6 with Retry

1.2 Flit Structure Description

A complete 256-byte flit is structured as shown below. Each region serves a distinct purpose within the transmission pipeline.

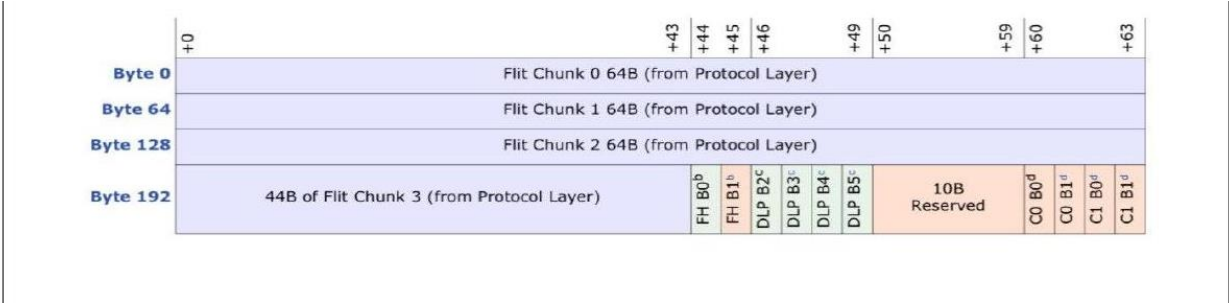


Figure 1-2: 256-Byte Flit Layout

Offset	Content
Byte 0	Flit Chunk 0 — 64B (from Protocol Layer)
Byte 64	Flit Chunk 1 — 64B (from Protocol Layer)
Byte 128	Flit Chunk 2 — 64B (from Protocol Layer)
Byte 192	44B of Flit Chunk 3 FH B0 FH B1 DLP B2–B5 10B Reserved CRC0 CRC1

Table 1-1: 256-Byte Flit Field Summary

The individual flit fields are described as follows:

Flit Chunks 0–2	Three contiguous 64-byte regions (192 bytes total) carrying unmodified payload data from the Protocol Layer.
Flit Chunk 3	The remaining 44-byte portion of the protocol payload required to complete the 256-byte flit size. Included in CRC computation.
Flit Header (FH B0, FH B1)	Two bytes carrying control metadata: protocol type, sequence number, and Ack/Nak information when retry is enabled.
DLP Bytes (B2–B5)	Four bytes used by the Adapter to convey DLLP and Optimized Update Marker information.
10B Reserved	Reserved bytes, always driven to zero. Kept for format consistency and future protocol extensions.
CRC Bytes (CRC0 & CRC1)	Two 16-bit CRC values generated by the Adapter to guarantee data integrity. Validated by the receiver to detect transmission errors.

2. MainBand Block

2.1 Mainband Block Overview

The Mainband is the core data-path block within the UCle Adapter. It handles flit-level processing in both the transmit (TX) and receive (RX) directions, bridging the Protocol Layer and the physical interface.

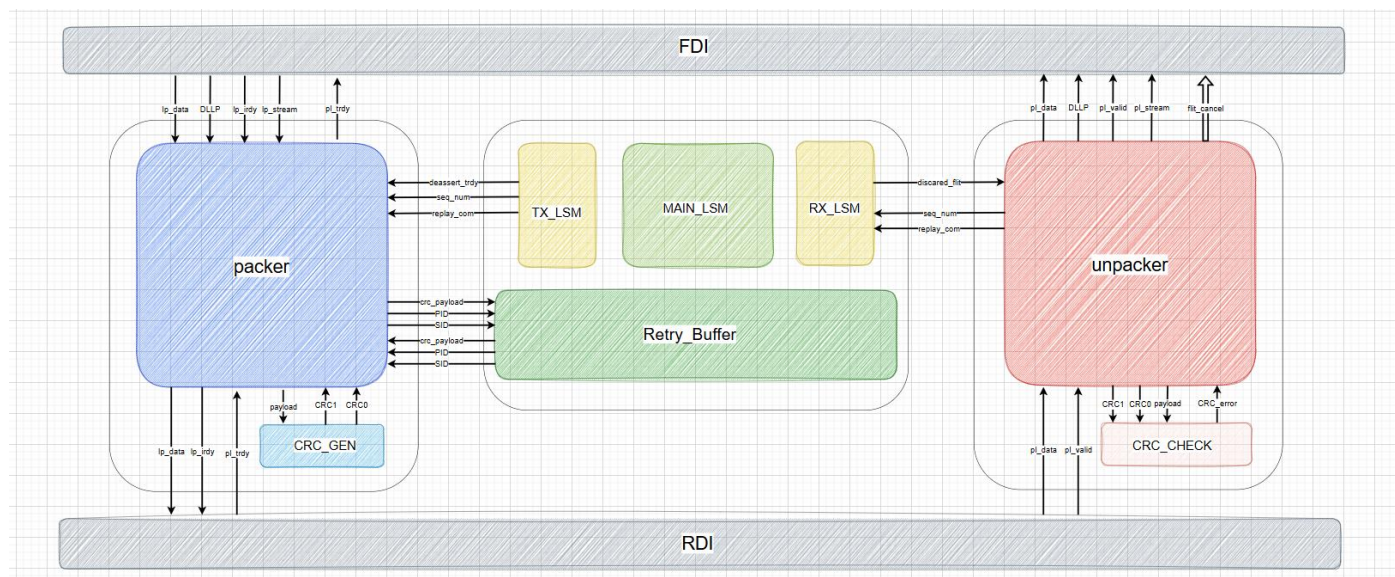


Figure 2-1: Mainband Block Diagram

Its primary responsibilities include:

- Constructing and validating 256-byte flits in accordance with the UCle protocol specification.
- Inserting and verifying CRC values to ensure end-to-end data integrity.
- Managing sequence numbers and replay commands to support the retry mechanism.
- Forwarding control signals and payload data across the FDI and RDI interfaces.

The Mainband is composed of the following sub-blocks:

Sub-Block	Function
Packer	Assembles outgoing flits from protocol payload and control fields.
Retry Buffer	Stores transmitted flits for potential replay in the event of a detected error.
CRC Generator	Computes CRC0 and CRC1 over the assembled flit for error protection.
Unpacker	Decomposes incoming flits into payload, control, and diagnostic fields.
CRC Check	Recomputes and verifies CRC values on received flits to detect corruption.

2.2 TX Path

In the transmit direction, protocol payload enters through the FDI interface and is processed by the Packer. The CRC Generator computes integrity values over the assembled flit, which are then appended before transmission via the RDI interface.

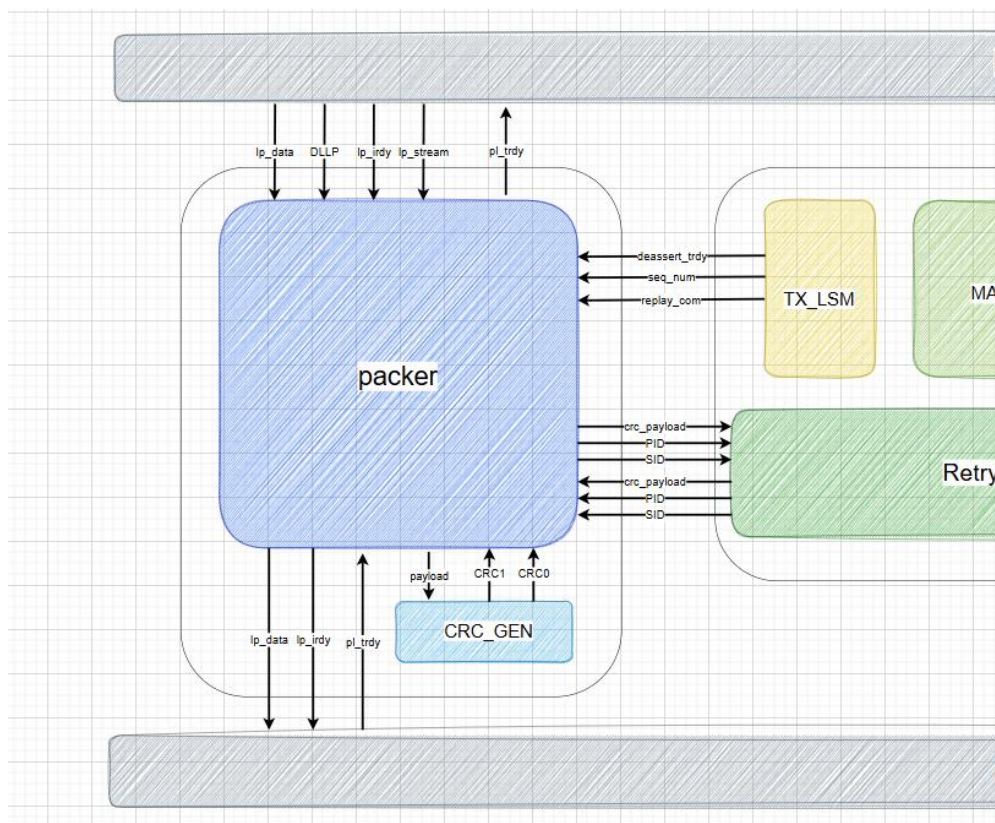


Figure 2-2: TX Path Block Diagram

2.3 RX Path

In the receive direction, incoming flit data arrives via the RDI interface and is processed by the Unpacker. The CRC Check block validates data integrity by recomputing and comparing CRC values.

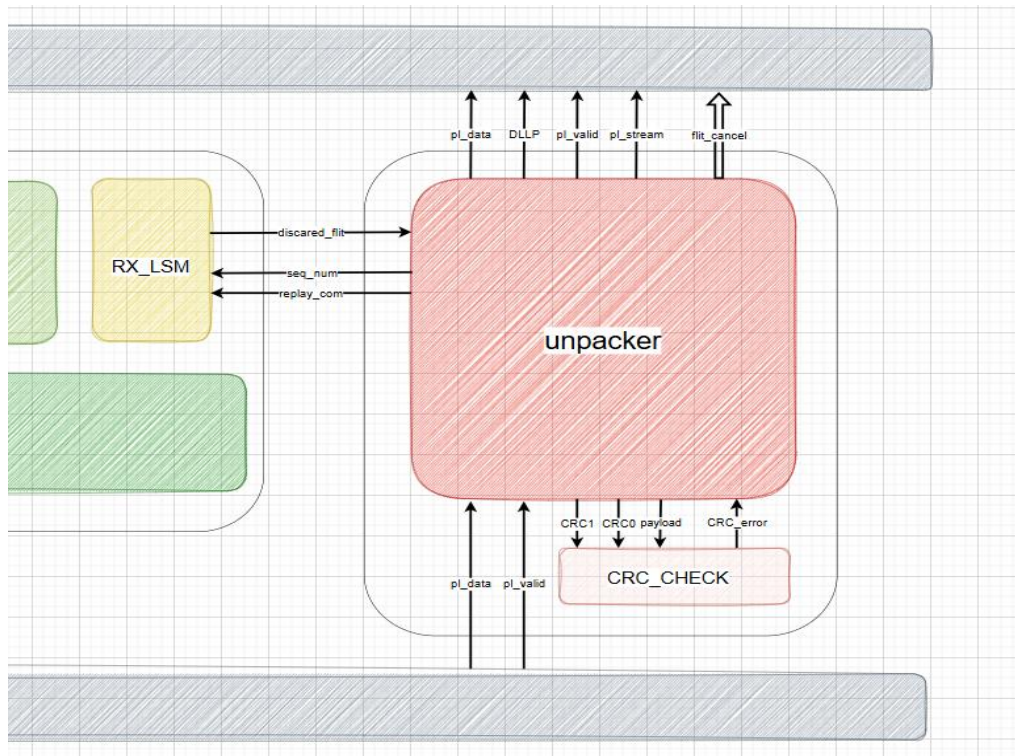


Figure 2-3: RX Path Block Diagram

3. Packer

3.1 Overview

The Packer operates exclusively in the transmit path and is responsible for constructing outgoing flits. It receives payload data from the Protocol Layer through the FDI interface and prepares complete flit structures for transmission.

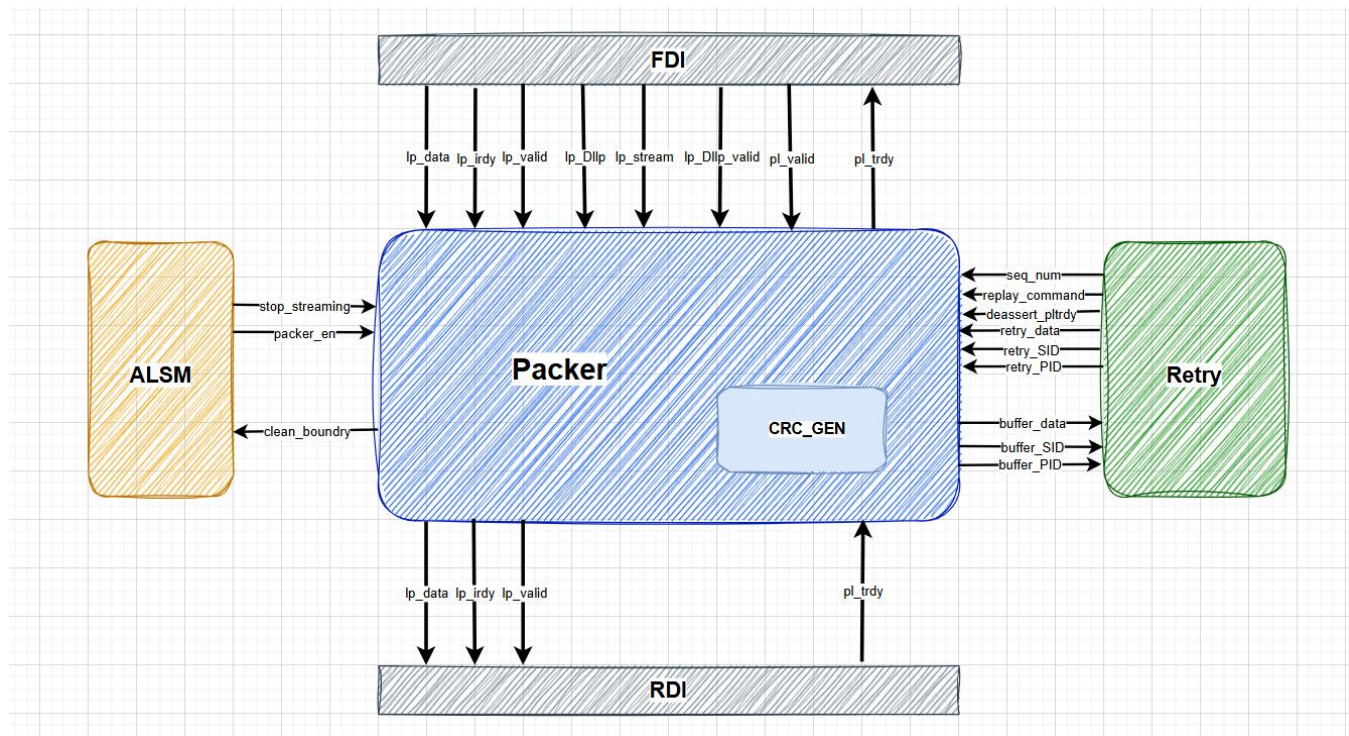


Figure 3-1: Packer Block Diagram

The Packer assembles the flit header using control metadata including the Protocol ID, Stack ID, sequence number, replay command, and flit type. The resulting header is combined with the payload data and any required control fields to form a complete flit, prior to CRC insertion by the CRC Generator.

By enforcing correct format and byte alignment, the Packer ensures protocol compliance and enables reliable, deterministic transmission behavior.

3.2 Signal Description

Port	Direction	Width	Connected	Description
Lp_irdy_fdi	IN	1	FDI	Indicates that the protocol layer is ready to send data.
Lp_valid_fdi	IN	1	FDI	Indicates that data from FDI is valid.
Lp_data_fdi	IN	512	FDI	Payload data received from the Protocol Layer via FDI.
Lp_dllp	IN	32	FDI	DLLP or optimized flow control information from FDI.
Lp_dllp_valid	IN	1	FDI	Indicates that the DLLP data is valid.
Lp_dllp_ofc	IN	1	FDI	Indicates that the DLLP carries optimized flow control info.
Lp_stream	IN	8	FDI	Contains Protocol ID and Stack ID information.
Pl_trdy	IN	1	RDI	Indicates that the Adapter is ready to accept a flit.
Packer_en	IN	1	LSM	Enables the packer operation in the TX path.
Stop_stream	IN	1	LSM	Stops data transmission from the protocol layer.
Seq_num	IN	8	Retry	Sequence number assigned to the current flit.
Replay_command	IN	2	Retry	Indicates replay or retry control command.
Deassert_trdy	IN	1	Retry	Requests deassertion of ready during retry handling.
Retry_data	IN	512	Retry	Payload data retrieved from retry buffer during replay.
Retry_SID	IN	1	Retry	Stack ID retrieved from retry buffer.
Retry_PID	IN	2	Retry	Protocol ID retrieved from retry buffer.
Pl_trdy_fdi	OUT	1	FDI	Indicates that the PHY is ready to receive data from Adapter.
Clean_boundry	OUT	1	LSM	Indicates a clean flit boundary after transmission.
Lp_data_rdi	OUT	512	RDI	Complete flit data sent to the RDI interface.
Lp_valid_rdi	OUT	1	RDI	Indicates that the flit data sent to RDI is valid.
Lp_irdy_rdi	OUT	1	RDI	Indicates that the packer is ready to transmit to RDI.
Buffer_data	OUT	512	Retry	Payload data stored for possible retry operation.
Buffer_PID	OUT	1	Retry	Protocol ID stored in the retry buffer.
Buffer_SID	OUT	1	Retry	Stack ID stored in the retry buffer.

3.3 TX Path Operation

The TX path begins by waiting for an enable assertion from the LSM. Once enabled, the Mainband asserts `pl_trdy` and waits until both `lp_irdy` and `lp_valid` are simultaneously asserted, completing the handshake.

Upon handshake completion, `lp_data` is received from the FDI interface and stored in an internal buffer. In parallel, `lp_stream` and `lp_dllp` signals are captured in dedicated buffers. The payload data, Protocol ID, and Stack ID are forwarded to the Retry block for tracking and potential replay handling.

The Mainband then receives the sequence number and replay command from the Retry block, at which point it constructs the Flit Header. During a retry, payload and IDs are retrieved from the Retry block while DLLP information continues to be sourced from FDI. The complete flit — header, payload, and CRC — is then transmitted to the RDI interface.

4. CRC Generator

4.1 Overview

The CRC Generator protects transmitted data against corruption by computing two independent CRC values — CRC0 and CRC1 — over fixed, non-overlapping windows of the flit.

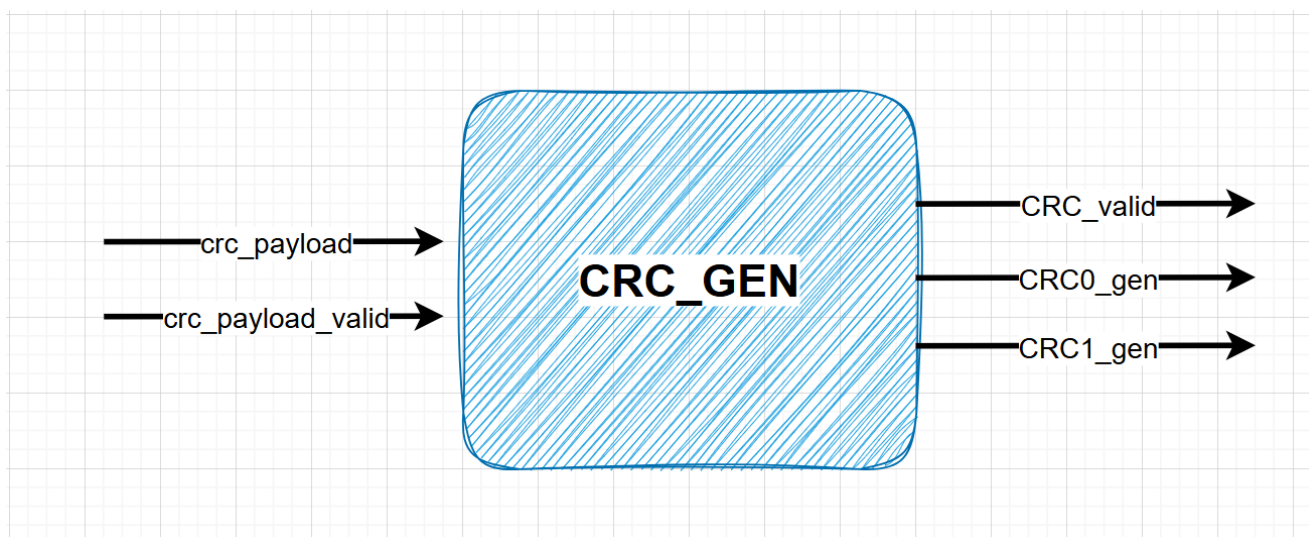


Figure 4-1: CRC Generator Block Diagram

It receives the partially assembled flit from the Packer and includes the flit header, payload data, and any inserted DLLP bytes in its computation, while explicitly excluding the CRC fields themselves. The computed values are returned to the Packer for insertion into the final flit.

4.2 Signal Description

Port	Direction	Width	Connected	Description
CLK	IN	1	—	Clock input.
RST	IN	1	—	Reset input.
crc_payload_valid	IN	1	Packer	Indicates valid data for CRC computation.
crc_payload	IN	512	Packer	Flit data excluding CRC fields, used for CRC calculation.
crc0_gen	OUT	16	Packer	CRC value calculated for the first 128 bytes of the flit.
crc1_gen	OUT	16	Packer	CRC value calculated for the second CRC window.
crc_valid	OUT	1	Packer	Indicates that CRC calculation is complete and valid.

4.3 CRC-16 Architecture (64 Bytes per Clock)

The CRC block implements a 16-bit Cyclic Redundancy Check using the generator polynomial $x^{16} + x^{15} + x^2 + 1$ (0x1021). The design processes 64 bytes (512 bits) per clock cycle, making it well-suited for UCle 256-byte flit processing.

Rather than processing one bit per cycle, this design unrolls the LFSR bit-by-bit logic into a single combinational function that processes all 512 bits in parallel. During synthesis, this produces a large parallel XOR network capable of updating the CRC register in a single clock cycle. Since each cycle handles 64 bytes, a full 256-byte flit requires four clock cycles to complete CRC computation.

4.4 CRC Generator Operation

CRC generation proceeds immediately upon receiving data from the Packer. CRC0 covers the first 128 bytes of the flit; CRC1 covers the remaining 114 bytes of valid flit data. In both windows, the size is fixed and unused bytes are treated as zero. Upon completion, CRC0 and CRC1 are forwarded to the Packer and `crc_valid` is asserted.

5. Unpacker

5.1 Overview

The Unpacker operates in the receive path and processes incoming flits from the RDI interface. It buffers received data and decomposes each flit into its constituent components for independent handling.

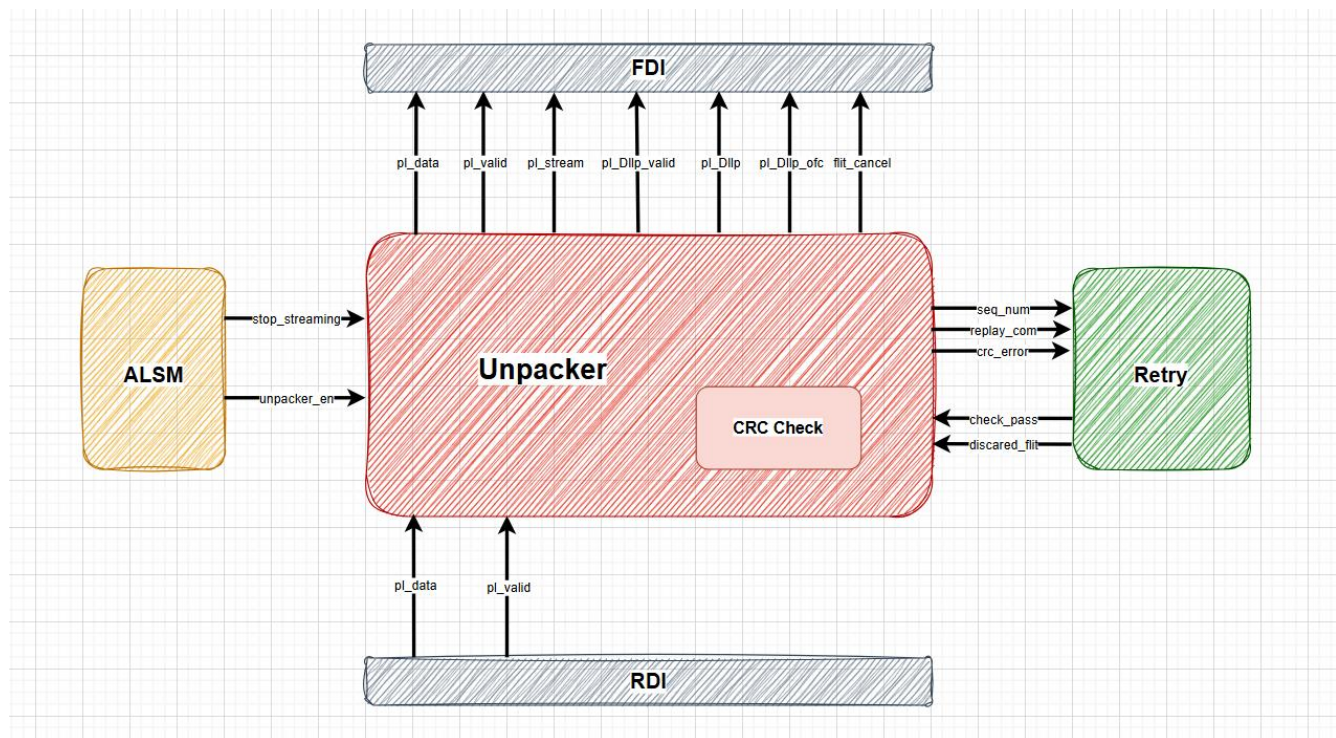


Figure 5-1: Unpacker Block Diagram

Extracted fields include the payload data, Protocol ID, Stack ID, DLLP information, sequence number, and replay command. By cleanly separating data and control information, the Unpacker enables correct protocol-layer interpretation and ensures valid payload delivery via the FDI interface.

5.2 Signal Description

Port	Direction	Width	Connected	Description
PI_datd_rdi	IN	512	RDI	Full received flit data including header, payload, DLLP, and CRC.
PI_valid_rdi	IN	1	RDI	Indicates that incoming flit data is valid.
Check_pass	IN	1	Retry	Asserted by Retry block when sequence and replay checks pass.
Discard_flit	IN	1	Retry	Indicates that the current flit must be discarded due to retry failure.
Stop_stream	IN	1	LSM	Signal from LSM to stop unpacking and data forwarding.
Unpacker_en	IN	1	LSM	Enables the Unpacker to start processing incoming flits.
PI_data_fdi	OUT	512	FDI	Valid payload data forwarded to the FDI interface.
PI_valid_fdi	OUT	1	FDI	Indicates that the output payload data to FDI is valid.
PI_stream	OUT	8	FDI	Carries Protocol ID and Stack ID information to FDI.
PI_dllp	OUT	32	FDI	Extracted DLLP data forwarded to the FDI interface.
PI_dllp_valid	OUT	1	FDI	Indicates that the DLLP data is valid.
PI_dllp_ofc	OUT	1	FDI	Indicates that the DLLP carries optimized flow control information.
Flit_cancel	OUT	1	FDI	On CRC or sequence error, instructs FDI to discard the forwarded payload.
Seq_num	OUT	8	Retry	Sequence number extracted from the flit header, sent to Retry block.
Replay_com	OUT	2	Retry	Replay command extracted from the flit header for Retry handling.
Crc_err	OUT	1	Retry	Asserted when a CRC mismatch is detected in the received flit.

5.3 RX Path Operation

When `pl_valid` is asserted and the Unpacker is enabled, it begins receiving `pl_data` from the RDI interface. The received data is stored in an internal buffer and decomposed into its constituent fields. The Stack ID and Protocol ID are extracted and driven on `pl_stream`; DLLP information is forwarded on `pl_dllp`; sequence number and replay command are sent to the Retry block.

CRC checking is initiated in parallel by recomputing CRC values over the received flit, excluding the CRC fields. If `flit_cancel` is asserted, FDI discards the forwarded payload.

5.4 Flit Cancel Mechanism

To minimize end-to-end latency, the receiver employs a cut-through forwarding architecture. Each 64-byte chunk is forwarded to the upper layer as soon as it is received (with a one-cycle pipeline delay), while the CRC engine and retry sequence checker operate in parallel.

The final validation results become available immediately after the last chunk is processed. If both checks pass, the flit is accepted. If a CRC error or sequence error is detected, a Flit Cancel signal is transmitted, instructing the receiving protocol layer to invalidate and discard the previously forwarded flit data.

6. CRC Check

6.1 Overview

The CRC Check block verifies the integrity of received flits by recomputing CRC0 and CRC1 using the same fixed windows applied during transmission and comparing them against the embedded CRC values.

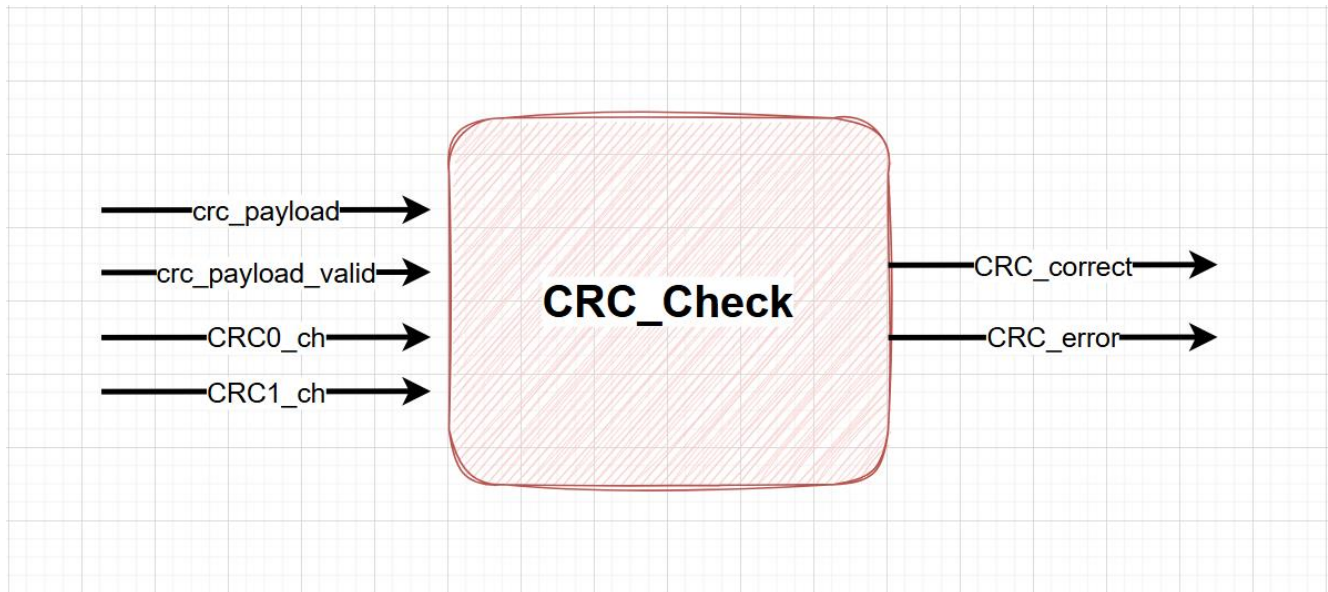


Figure 6-1: CRC Check Block Diagram

A successful match results in assertion of `crc_correct`. A mismatch causes the block to assert `crc_error`, preventing corrupted data from propagating and triggering the retry and recovery mechanisms.

6.2 Signal Description

Port	Direction	Width	Connected	Description
CLK	IN	1	—	Clock input.
RST	IN	1	—	Reset input.
crc_payload	IN	512	Unpacker	Flit data excluding CRC fields, used as input to CRC calculation.
crc_payload_valid	IN	1	Unpacker	Indicates that the received flit data is valid for CRC checking.
crc0_ch	IN	16	Unpacker	CRC0 value received within the flit.
crc1_ch	IN	16	Unpacker	CRC1 value received within the flit.
crc_correct	OUT	1	Unpacker	Asserted when calculated CRC matches received CRC values.
crc_err	OUT	1	Unpacker	Asserted when a CRC mismatch is detected.

6.3 CRC Check Operation

Incoming data is processed to recompute CRC0 and CRC1 over the same fixed windows used during transmission. The received CRC0 and CRC1 values are supplied concurrently by the Unpacker and compared against the recomputed results.

If both values match, `crc_correct` is asserted, confirming the flit was received without corruption. If a mismatch is detected, `crc_error` is forwarded to the Retry block and a stop signal is asserted to the Unpacker, halting further data forwarding until the error condition is resolved.

7. Implementation

Architecture & Design Reference

Author: Ali Nouredin Abdelaziz

Project: UCle 3.0 Adapter Layer — Mainband Block

Language: SystemVerilog

1. Project Overview

The Mainband block is the core of the UCle 3.0 Adapter Layer, responsible for flit-level data processing in both TX and RX paths. It sits between the Protocol Layer (FDI interface) and the Physical Layer (RDI interface).

1.1 Sub-Block Summary

Block	Path	Function
Packer	TX	Assembles 256B flits, computes CRC, sends to RDI
Unpacker	RX	Receives flits from RDI, checks CRC, forwards to FDI
CRC_Generator	TX + RX	16-bit CRC computation (LFSR-based, 512 bits/cycle)

2. Flit Format (Format 3 — 256B)

A complete 256-byte flit is laid out across four 64-byte chunks. The final chunk carries the protocol payload, flit header, DLP bytes, reserved region, and two 16-bit CRC values.

2.1 Byte Map

Region	Byte Range	Size	Description
Chunk 0	0 – 63	64 B	Payload from Protocol Layer
Chunk 1	64 – 127	64 B	Payload from Protocol Layer
Chunk 2	128 – 191	64 B	Payload from Protocol Layer
Chunk 3 Payload	192 – 235	44 B	Remaining protocol payload
FH B0 / B1	236 – 237	2 B	Flit Header bytes
DLP B2–B5	238 – 241	4 B	DLLP / Optimized Update Marker
Reserved	242 – 251	10 B	Always zero; reserved for future use
CRC0	252 – 253	2 B	16-bit CRC over first 128 bytes
CRC1	254 – 255	2 B	16-bit CRC over second window

2.2 Chunk 3 Bit-Field Offsets (within 512-bit chunk)

Field	Bits	Size
Payload	[351:0]	44 B
FH Byte 0	[359:352]	1 B
FH Byte 1	[367:360]	1 B
DLP B2–B5	[399:368]	4 B
Reserved	[479:400]	10 B
CRC0	[495:480]	2 B
CRC1	[511:496]	2 B

2.3 Flit Header Encoding

Byte	Bits	Field	Description
FH_B0	[7:6]	PID	Protocol Identifier
FH_B0	[5]	SID	Stack Identifier
FH_B0	[4]	OFC	Optimized Flow Control flag
FH_B0	[3:0]	SEQ[7:4]	Upper 4 bits of sequence number
FH_B1	[7:6]	Flit_Type	00b = normal flit
FH_B1	[5:4]	Ack/Nak	Retry acknowledgment / negative-ack
FH_B1	[3:0]	SEQ[3:0]	Lower 4 bits of sequence number

3. CRC Generator

The CRC Generator computes two independent 16-bit CRC values — CRC0 and CRC1 — over fixed, non-overlapping windows of the 256-byte flit. The polynomial used is $x^{16} + x^{15} + x^2 + 1$ (0x1021).

3.1 Architecture & Timing

- LFSR-based implementation, processes 512 bits (64 bytes) per clock cycle.
- Full 256-byte flit requires four clock cycles to complete CRC computation.
- CRC fields are zeroed before being fed to the generator (excluded from computation).
- CRC0 covers the first 128 bytes; CRC1 covers the second 128-byte window.

3.2 Cycle-by-Cycle Timing

Cycle	State	Action
1	S_CHUNK0	Receive / process Chunk 0 — begin CRC0 computation
2	S_CHUNK1	Receive / process Chunk 1 — CRC0 window complete
3	S_CHUNK2	Receive / process Chunk 2 — begin CRC1 computation
4	S_CHUNK3	Receive / process Chunk 3 — assert o_crc_valid

4. Packer (TX Path)

The Packer operates exclusively in the transmit path, assembling 256-byte flits from protocol-layer payload chunks. It feeds the CRC Generator in parallel and delivers completed flits to the RDI interface with a total pipeline latency of five clock cycles.

4.1 Pipeline Timing

Cycle	Action
1	Receive Chunk 0 from FDI; feed to CRC Generator
2	Receive Chunk 1; feed CRC Gen Chunk 0 → RDI
3	Receive Chunk 2; feed CRC Gen Chunk 1 → RDI
4	Receive Chunk 3 (masked); feed CRC Gen Chunk 2 → RDI
5	CRC ready — insert FH + DLP + Reserved + CRC into Chunk 3; Chunk 3 → RDI

4.2 FSM State Descriptions

State	Description
S_IDLE	Wait for packer_en assertion from LSM
S_COLLECT	Cycles 1–4: receive chunks, feed CRC, pipeline send to RDI
S_INSERT	Cycle 5: insert FH, DLP, reserved, CRC into Chunk 3; send to RDI
S_DONE	Check pending commands; prepare next flit
S_FLIT_BOUNDARY	Assert flit_boundary_done; return to S_IDLE
S_DRAIN	Send retry buffer with replay command until buffer is empty
S_FLUSH	Send retry buffer with replay_cmd forced to 00

4.3 Data Source per Mode

Mode	Data Source	replay_cmd
Normal	FDI	Real value from LSM
Retry	Retry buffer	Real value
Drain	Retry buffer	Real value
Flush	Retry buffer	Forced to 00

4.4 NOP Handling

- When `lp_valid = 0`, the Packer stores zeros and marks the chunk as NOP (`r_nop_chunk[i] = 1`).
- NOP chunks suppress Flit Header insertion in Chunk 3.
- When `lp_dllp_valid = 0`, the DLP bytes are zeroed.

4.5 Key Internal Signals

Signal	Type	Description
<code>r_collect_cnt</code>	reg [1:0]	Chunk counter (0→3)
<code>r_nop_chunk</code>	reg [3:0]	NOP flag per chunk
<code>r_pipe_data</code>	reg [511:0]	Pipeline register (1-cycle delay)
<code>r_pipe_valid</code>	reg	Pipeline data valid flag
<code>r_chunk3_buf</code>	reg [511:0]	Holds Chunk 3 until <code>S_INSERT</code>
<code>r_flush_mode</code>	reg	Forces <code>replay_cmd = 00</code> in flush mode
<code>r_flit_boundary_pending</code>	reg	Latches <code>flit_boundary</code> assertion mid-flit
<code>r_drain_pending</code>	reg	Latches drain command mid-flit
<code>r_flush_pending</code>	reg	Latches flush command mid-flit
<code>w_chunk3_masked</code>	wire [511:0]	Chunk 3 with CRC fields zeroed for <code>CRC_Gen</code>
<code>w_fh_b0 / w_f_b1</code>	wire [7:0]	Assembled flit header bytes

5. Unpacker (RX Path)

The Unpacker operates in the receive path, processing incoming 256-byte flits from the RDI interface. It employs cut-through forwarding to minimise end-to-end latency while running CRC verification in parallel.

5.1 Pipeline Timing

Cycle	Action
1	Receive Chunk 0; feed to CRC Generator
2	Receive Chunk 1; feed CRC Gen Chunk 0 → FDI
3	Receive Chunk 2; feed CRC Gen Chunk 1 → FDI
4	Receive Chunk 3 (masked); feed CRC Gen Chunk 2 → FDI
5	S_CHECK — CRC compare + forward Chunk 3 to FDI + assert flit_cancel if error

5.2 FSM State Descriptions

State	Description
S_START	Wait for unpacker_en assertion from LSM
S_RECEIVE	Cycles 1–4: receive chunks, feed CRC Generator, cut-through forward to FDI
S_CHECK	Cycle 5: CRC comparison; forward Chunk 3; assert flit_cancel on error

5.3 Chunk 3 Field Extraction

Register	Source Bits	Field Extracted
r_pid	FH_B0[7:6]	Protocol Identifier (PID)
r_sid	FH_B0[5]	Stack Identifier (SID)
r_dllp_ofc	FH_B0[4]	Optimized Flow Control flag
r_replay_cmd	FH_B1[5:4]	Ack / Nak replay command
r_seq_num	{FH_B0[3:0], FH_B1[3:0]}	Full 8-bit sequence number
r_dllp_buf	i_pl_data_rdi[C3_DLP +: 32]	Extracted DLLP bytes
r_crc0_ch	i_pl_data_rdi[C3_CRC0 +: 16]	Received CRC0
r_crc1_ch	i_pl_data_rdi[C3_CRC1 +: 16]	Received CRC1

5.4 Key Internal Signals

Signal	Type	Description
r_chunk_cnt	reg [1:0]	Received chunk counter (0→3)
r_pipe_data	reg [511:0]	Pipeline register (1-cycle delay)
r_pipe_valid	reg	Pipe valid — prevents garbage on first cycle
r_crc0_ch	reg [15:0]	Received CRC0 from flit
r_crc1_ch	reg [15:0]	Received CRC1 from flit
w_chunk3_masked	wire [511:0]	Chunk 3 with CRC fields zeroed for CRC_Gen

6. Mainband Top-Level

The Mainband top-level module (Mainband.sv) instantiates the Packer and Unpacker sub-blocks and exposes LSM and Retry signals as top-level ports.

Instance	Sub-Block	Interface
U1_Packer	Packer (TX)	FDI TX inputs + RDI TX outputs
U2_Unpacker	Unpacker (RX)	RDI RX inputs + FDI RX outputs

7. Key Design Decisions

Decision	Reason
5-cycle pipeline for Packer	Feeding the CRC Generator in parallel with chunk collection saves four or more cycles compared to a sequential approach.
Cut-through forwarding in Unpacker	Each 64-byte chunk is forwarded to the upper layer one cycle after receipt, reducing latency significantly versus store-and-forward.
CRC fields zeroed before CRC_Gen	Required by the UCle specification; CRC fields must be excluded from the CRC computation.
Flit Header inserted only in Chunk 3	Header bytes FH_B0 and FH_B1 are located at bytes 236–237, which fall within Chunk 3.
r_flush_mode flag	A single register differentiates flush from drain mode inside S_COLLECT, keeping the FSM compact.
NOP chunk tracking (r_nop_chunk)	Per-chunk NOP flags prevent inadvertent Flit Header insertion for chunks where lp_valid was de-asserted.
Retry buffer written only in normal mode	Data is already stored in the retry buffer from the prior transmission cycle.
Pipeline register (r_pipe_data)	Required because the first cycle has no previous chunk to forward; gated by r_pipe_valid to suppress garbage.

8. Naming Conventions

Prefix	Meaning
r_	Register (flip-flop, sequential logic)
w_	Wire (combinational logic)
i_	Input port
o_	Output port

9. Source Files

File	Description
Mainband_pkg.sv	Package containing all parameters and typedefs
CRC_Generator.sv	16-bit CRC computation block
Packer.sv	TX path — flit assembly and header insertion
Unpacker.sv	RX path — flit decomposition and CRC verification
Mainband.sv	Top-level module connecting Packer and Unpacker
Packer_tb.sv	Testbench for Packer (7 test cases)
Unpacker_tb.sv	Testbench for Unpacker (7 test cases)

10. Testbench Test Cases

10.1 Packer Testbench

TC	Name	Description
1	Normal TX	Real data with real DLLP; verify correct flit assembly
2	NOP Data	Chunks 1 and 2 with lp_valid = 0; verify NOP handling
3	NOP DLLP	All chunks valid but dllp_valid = 0; DLP bytes must be zero
4	Replay Mode	Data sourced from retry buffer; verify replay path
5	Flit Boundary	Assert flit_boundary mid-flit; verify boundary handshake
6	Drain	Send retry buffer until empty; verify drain state
7	Flush	Send retry buffer with replay_cmd = 00; verify flush mode

10.2 Unpacker Testbench

TC	Name	Description
1	Normal RX	Valid flit with correct CRC — flit_cancel must not assert
2	CRC Error	Corrupted CRC — verify crc_err = 1 and flit_cancel = 1
3	Cut-Through	Verify Chunk 0 appears on FDI output at cycle 2
4	Sequence Error	discarded_flit asserted — flit_cancel must assert
5	Stop Stream	stop_stream mid-receive — verify return to S_START
6	DLLP Extraction	Verify o_pl_dllp matches DLP bytes in received flit
7	Consecutive Flits	Two back-to-back flits with no inter-flit gap

8. Link

https://drive.google.com/drive/folders/1kp7jUQAgoN8ChLGmv6ptrXPA83bGhhwC?usp=drive_link

End of Document — UCle Final System for Mainband